CSA-0903 PROGRAMING IN JAVA

SCENARIO BASED

P.C. BHARATH SIMHA REDDY

192424361

1. Single Inheritance – Scenario

Scenario:

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car class extends Vehicle and adds a playMusic() method.

The company now wants every car to automatically log the engine start time whenever startEngine() is called, while still using the original engine-start functionality.

Questions:

Q1.How would you implement this using single inheritance?

Answer:

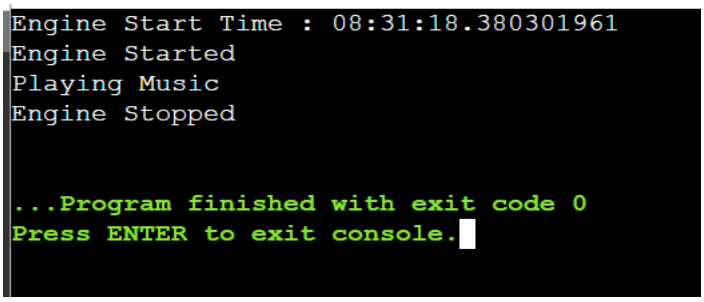
Create a parent class Vehicle containing startEngine() and stopEngine(). Then create a child class Car using the extends keyword. The Car class inherits all methods from Vehicle and adds its own method playMusic(). To log the engine start time, override the startEngine() method in Car.

CODE:

```java
import java.time.LocalTime;
class Vehicle {

    void startEngine() {
        System.out.println("Engine Started");
    }
    void stopEngine() {
        System.out.println("Engine Stopped");
    }
}
class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Engine Start Time : " + LocalTime.now());
        super.startEngine();
    }
    void playMusic() {
        System.out.println("Playing Music");
    }
}
public class Main {
    public static void main(String args[]) {
        Car c = new Car();
        c.startEngine();
        c.playMusic();
        c.stopEngine();
    }
}
```

OUTPUT:



Q2.Should Car override startEngine()? Why?

Answer:

Yes. Car should override startEngine() because it needs to perform additional work (logging the engine start time) while still using the original functionality of the Vehicle class.

Q3. How can the parent class implementation still be executed?

Answer:

The parent class method can be executed using the super keyword.

super.startEngine();

This calls the original startEngine() method of the Vehicle class.

Q4. What happens if Car does not override startEngine()?

Answer:

If Car does not override startEngine(), it automatically inherits the method from Vehicle. The engine starts normally, but the engine start time is not logged.

2. Multilevel Inheritance – Scenario

Scenario:

A software company models employees as follows:

- Employee → stores employee ID and name.
- Developer extends Employee → stores programming language.
- SeniorDeveloper extends Developer → stores team size and performs code reviews.

Now HR requests that every SeniorDeveloper should be able to access employee information and programming language details without duplicating code.

Questions:

Q1.Why is multilevel inheritance suitable here?

Answer:

Multilevel inheritance is suitable because every class adds new features while inheriting the existing features of the previous class.

- Employee stores employee details.
- Developer inherits employee details and adds programming language.
- SeniorDeveloper inherits both employee details and programming language and adds team size and code review functionality.

This avoids duplicate code and improves reusability.

CODE:

```
class Employee {
    int empId = 101;
    String name = "Rahul";
    void displayEmployee() {
        System.out.println("Employee ID : " + empId);
        System.out.println("Employee Name : " + name);
    }
}
class Developer extends Employee {
    String language = "Java";
    void displayLanguage() {
        System.out.println("Programming Language : " + language);
    }
}
class SeniorDeveloper extends Developer {
    int teamSize = 8;
    void codeReview() {
        System.out.println("Code Review Completed");
        System.out.println("Team Size : " + teamSize);
    }
}
public class Main {
    public static void main(String args[]) {
        SeniorDeveloper s = new SeniorDeveloper();
        s.displayEmployee();
        s.displayLanguage();
        s.codeReview();
    }
}
```

OUTPUT:

```
Employee ID : 101
Employee Name : Rahul
Programming Language : Java
Code Review Completed
Team Size : 8


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. Which members are accessible in SeniorDeveloper?

Answer:

SeniorDeveloper can access:

- Employee ID
- Employee Name
- Employee methods
- Programming Language
- Developer methods
- Team Size
- Code Review method

Q3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

Answer:

SeniorDeveloper inherits the overridden method from the Developer class because it is the nearest parent.

Q4. Draw the inheritance hierarchy.

Employee

|

Developer

|

SeniorDeveloper

3. Hierarchical Inheritance – Scenario

Scenario:

An online shopping application has a base class Payment.

Three classes inherit from it:

- CreditCardPayment
- UPIPayment
- NetBankingPayment

Each payment type processes transactions differently but shares common validation logic.

Questions:

Q1.Why is hierarchical inheritance appropriate here?

Answer:

Hierarchical inheritance is appropriate because multiple classes (CreditCardPayment, UPIPayment, and NetBankingPayment) inherit from a single parent class Payment. This allows all payment types to share common features while implementing their own payment processing.

CODE:

```
class Payment {
    void validate() {
        System.out.println("Validation Successful");
    }
    void processPayment() {
        System.out.println("Processing Payment");
    }
}
class CreditCardPayment extends Payment {
    @Override
    void processPayment() {
        System.out.println("Credit Card Payment Completed");
    }
}
class UPIPayment extends Payment {
    @Override
    void processPayment() {
        System.out.println("UPI Payment Completed");
    }
}
class NetBankingPayment extends Payment {
    @Override
    void processPayment() {
        System.out.println("Net Banking Payment Completed");
    }
}
public class Main {
    public static void main(String args[]) {
        CreditCardPayment c = new CreditCardPayment();
        c.validate();
        c.processPayment();
```

OUTPUT:

```
Validation Successful
Credit Card Payment Completed


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. Where should common validation logic be implemented?

Answer:

The common validation logic should be implemented in the parent class Payment because all payment methods require the same validation process.

Q3. How can each payment class provide its own transaction processing?

Answer:

Each payment class should override the processPayment() method and implement its own transaction processing logic.

Q4. What is the advantage over writing three independent classes?

Answer:

- Reduces code duplication.
- Improves code reusability.
- Easier to maintain.
- Common functionality is written only once.
- Easy to extend by adding new payment methods.

4. Compile-Time Polymorphism (Method Overloading) – Scenario

Scenario:

A calculator application supports addition of:

- Two integers
- Three integers
- Two decimal numbers

The product owner wants users to call the same method name (add) regardless of the input.

Questions:

Q1.Which OOP concept should be used?

Answer:

Method Overloading (Compile-Time Polymorphism).

CODE:

```java
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {

    public static void main(String args[]) {

        Calculator c = new Calculator();

        System.out.println(c.add(5,10));
        System.out.println(c.add(5,10,15));
        System.out.println(c.add(5.5,10.5));
        System.out.println(c.add(5,10.0));

    }
}
```

OUTPUT:

```
15
30
16.0
15.0


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. How will Java decide which method to invoke?

Answer:

Java selects the appropriate overloaded method based on the number, type, and order of the arguments at compile time.

Q3. Can methods be overloaded by changing only the return type?

Answer:

No. Methods cannot be overloaded by changing only the return type because Java cannot differentiate methods using only their return values.

Q4. What happens if add(5,10.0) is called?

Answer:

Java automatically converts 5 to 5.0 and calls the add(double, double) method.

5. Runtime Polymorphism (Method Overriding) – Scenario

Scenario:

A food delivery application has:

Restaurant

|

| | |

Pizza Burger Biryani

Each restaurant calculates delivery time differently.

The application stores all restaurants in:

List<Restaurant> restaurants;

and calls:

restaurant.calculateDeliveryTime();

without checking the actual restaurant type.

Questions:

Q1.Which OOP concept is being used?

Answer:

Runtime Polymorphism (Method Overriding).

CODE:

```
class Restaurant {
    void calculateDeliveryTime() {
        System.out.println("General Delivery Time");
    }
}
class Pizza extends Restaurant {
    void calculateDeliveryTime() {
        System.out.println("Pizza Delivery : 30 Minutes");
    }
}
class Burger extends Restaurant {
    void calculateDeliveryTime() {
        System.out.println("Burger Delivery : 20 Minutes");
    }
}
class Biryani extends Restaurant {
    void calculateDeliveryTime() {
        System.out.println("Biryani Delivery : 40 Minutes");
    }
}
public class Main {
    public static void main(String args[]) {
        Restaurant r;
        r = new Pizza();
        r.calculateDeliveryTime();
        r = new Burger();
        r.calculateDeliveryTime();
        r = new Biryani();
        r.calculateDeliveryTime();
    }
}
```

OUTPUT:

```
Pizza Delivery : 30 Minutes
Burger Delivery : 20 Minutes
Biryani Delivery : 40 Minutes


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. Which version of calculateDeliveryTime() executes?

Answer:

The version belonging to the actual object (Pizza, Burger, or Biryani) executes.

Q3. When is the method selection decided?

Answer:

The method selection is decided at runtime using Dynamic Method Dispatch.

Q4. What would happen if calculateDeliveryTime() were declared static?

Answer:

If the method is declared static, it cannot be overridden. The method called depends on the reference type, not the object type.

6. Interface – Scenario

Scenario:

A smart home application supports different devices:

- Smart Bulb
- Smart Fan
- Smart AC

All devices can be turned on and turned off, but each device has its own implementation.

In the future, third-party manufacturers should also be able to integrate their own devices without modifying existing code.

Questions:

Q1.Would you use an interface or inheritance? Why?

Answer:

An **interface** should be used because all smart devices perform the same operations (turnOn() and

turnOff()), but each device has its own implementation. It also allows third-party manufacturers to create new devices without modifying existing code.

CODE:

```java
interface SmartDevice {
    void turnOn();
    void turnOff();
}
class SmartBulb implements SmartDevice {
    public void turnOn() {
        System.out.println("Bulb ON");
    }
    public void turnOff() {
        System.out.println("Bulb OFF");
    }
}
class SmartFan implements SmartDevice {
    public void turnOn() {
        System.out.println("Fan ON");
    }
    public void turnOff() {
        System.out.println("Fan OFF");
    }
}
public class Main {
    public static void main(String args[]) {
        SmartBulb b = new SmartBulb();
        b.turnOn();
        b.turnOff();
        SmartFan f = new SmartFan();
        f.turnOn();
        f.turnOff();

    }
}
```

OUTPUT:

```
Bulb ON
Bulb OFF
Fan ON
Fan OFF


...Program finished with exit code 0
Press ENTER to exit console.
```

Q2. Design an appropriate interface.

Answer:

interface SmartDevice {

    void turnOn();

```
    void turnOff();
}
```

Q3. Can one device implement multiple interfaces? Give an example.

Answer:

Yes. A class can implement multiple interfaces.

Example:

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
interface WiFiEnabled {
    void connectWiFi();
}
class SmartAC implements SmartDevice, WiFiEnabled {
    public void turnOn() {}
    public void turnOff() {}

    public void connectWiFi() {}
}
```

Q4. Why is an interface better than creating a common base class in this scenario?

Answer:

An interface is better because it defines a common contract without forcing a class hierarchy. It supports multiple interface implementation, provides flexibility, allows third-party integration, and makes the application easier to extend and maintain.